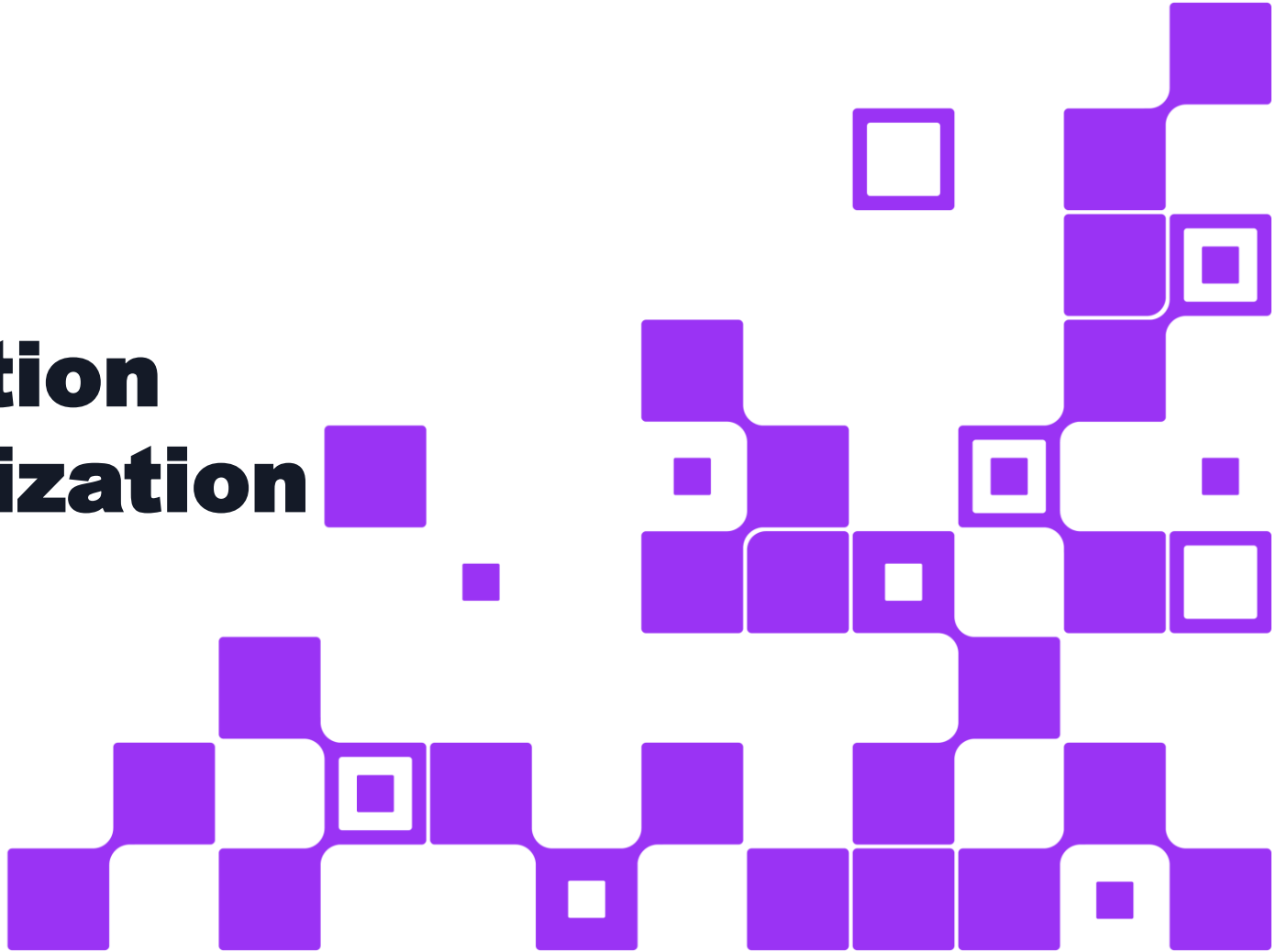
IThub

Serialization
Deserealization



IThub

Сериализация (Serialization) - это процесс записи состояния объекта в байтовый поток.

сохранить состояние программы в постоянном хранилище (файле)

кроссплатформенность – объект из потока байтов может быть восстановлен на любой ОС

если необходимо передать объект по сети

Средства сериализации позволяют сохранять и восстанавливать только объекты, реализующие интерфейс Serializable

Интерфейс Serializable — идентификатор, который не имеет методов, но он указывает jvm, что объекты этого класса могут быть сериализованы

Если класс сериализуемый, то все его подклассы тоже сериализуемые

transient

IThub

Для управления сериализацией при определении полей можно использовать ключевое слово **transient**, таким образом исключив поля из общего процесса сериализации.

Например, скрыть конфиденциальные данные, логин, пароль и тп.

Для выполнения процесса сериализации необходимо создать выходной поток OutputStream, упаковать его в **ObjectOutputStream** и вызвать метод writeObject().

Для восстановления (десериализации) объекта нужно упаковать InputStream в **ObjectInputStream** и вызвать метод readObject().

IThub

```java
public class Box implements Serializable {

    private double width;
    private double height;
    private double depth;

    protected transient String name;

    public Box(double width, double height,
      double depth, String name) {
        this.width = width;
        this.height = height;
        this.depth = depth;
        this.name = name;
    }
```

```java
public void serializableExampleDemo() {

    // сериализация
    try(ObjectOutputStream objOStream =
            new ObjectOutputStream(new FileOutputStream("box"))) {

        Box box = new Box(15, 45, 78, "My serializable box example");

        System.out.println("Object before serialization\n");
        System.out.println(box);

        objOStream.writeObject(box);

    } catch (IOException e) {
        System.out.println("Error during serialization: " + e.getMessage());
    }

    // десериализация
    try (ObjectInputStream objIStream
            = new ObjectInputStream(new FileInputStream("box"))) {

        Box disBox = (Box) objIStream.readObject();

        System.out.println("Object after deserialization\n");
        System.out.println(disBox);
    } catch (Exception e) {
        System.out.println("Error during deserialization: " + e.getMessage());
    }
}
```

Интерфейс Externalizable - это интерфейс, который позволяет полностью контролировать процесс сериализации и десериализации объекта.

IThub

Обычно Java использует интерфейс Serializable и сама сохраняет поля объекта

Externalizable делает иначе: разработчик сам пишет код, какие данные сохранять и как их восстанавливать

IThub

Интерфейс содержит два метода:

void writeExternal(ObjectOutput out) throws IOException
вызывается при сериализации. Здесь указываем, что записать в поток

void readExternal(ObjectInput in) throws IOException, ClassNotFoundException
вызывается при восстановлении (десериализации) объекта. Здесь читаем данные обратно.

```
class User implements Externalizable {
    private String name;
    private int age;
    private String login;

    public User() {
        // обязательно нужен public-конструктор без параметров
    }

    public User(String name, int age, String login) {
        this.name = name;
        this.age = age;
        this.login = login;
    }

    @Override
    public void writeExternal(ObjectOutput out) throws IOException {
        out.writeUTF(name);
        out.writeInt(age);
    }

    @Override
    public void readExternal(ObjectInput in) throws IOException {
        name = in.readUTF();
        age = in.readInt();
    }
}
```

Отличие от Serializable

IThub

Serializable	Externalizable
Java сама сохраняет поля	Разработчик сам пишет логику
Не требует методов	Нужно реализовать 2 метода
Проще использовать	Больше контроля
Не нужен конструктор без аргументов	Нужен public no-args constructor

Задание

IThub

Вы разрабатываете простую компьютерную игру.
Игрок проходит уровни, собирает монеты и получает оружие.
Нужно реализовать систему сохранения и загрузки игрового прогресса с помощью интерфейса Serializable.

Создайте класс Player, который хранит информацию об игроке (должен реализовывать интерфейс Serializable):

- String **nickname** — имя игрока
- int **level** — текущий уровень
- int **health** — здоровье игрока
- int **coins** — количество монет
- ArrayList<String> **inventory** — список предметов
- boolean **online** – несохраняемое поле

Алгоритм:

1) Создать игрока
2) Реализовать сохранение объекта игрока в файл
3) Реализовать загрузку сохранения из файла
4) После загрузки вывести информацию об игроке в консоль